



LABORATORIO 5

Introducción a Node.js y primer servidor web

1. Introducción

En este laboratorio se inicia el trabajo con **Node.js** para construir un servidor web básico. El objetivo es que los alumnos comprendan cómo funciona Node como entorno de ejecución, cómo responder a peticiones HTTP y cómo organizar el proyecto que dará soporte a la práctica obligatoria (gestión de reservas de vehículos eléctricos).

Se implementarán los primeros **endpoints estáticos y dinámicos**, que más adelante evolucionarán con Express.js y persistencia en bases de datos.

2. Objetivos del laboratorio

- Instalar y configurar un proyecto Node.js con npm init.
- Crear un servidor HTTP básico con Node.js.
- Servir contenido estático (HTML, CSS, imágenes).
- Implementar primeras rutas dinámicas (ej. /vehiculos).
- Manejar parámetros simples en la URL.
- Dejar sentada la estructura inicial del proyecto para migrar a Express en el siguiente laboratorio.

3. Tareas a realizar

Tareas básicas (obligatorias)

1. Crear un nuevo proyecto:
 - Carpeta raíz Lab5_Node.
 - Ejecutar npm init -y.
 - Instalar dependencias básicas si se consideran necesarias (ej. nodemon).
2. Implementar un servidor HTTP en app.js que:
 - Escuche en el puerto **3000**.
 - Responda en / con un HTML sencillo que incluya un mensaje de bienvenida al sistema de gestión de reservas.
 - Responda en /vehiculos con una página que muestre una lista estática de vehículos (3–5 ejemplos en un array).
3. Servir recursos estáticos (ej. CSS e imágenes) desde una carpeta public.

Tareas intermedias



4. Ampliar el endpoint /vehiculos para que acepte un **parámetro de consulta** (ej. /vehiculos?tipo=coche) y filtre la lista según ese valor.
5. Implementar una nueva ruta /reserva que muestre un formulario HTML simple para simular la reserva de un vehículo.
6. Manejar el método POST en /reserva para capturar los datos enviados y mostrarlos en consola.

Tareas avanzadas

7. Crear un **array en memoria** con reservas realizadas (nombre, vehículo, fecha). Cada vez que se envía el formulario se añade una entrada.
8. Implementar un endpoint /listareservas que muestre en HTML la lista acumulada de reservas.
9. Gestionar errores:
 - Ruta inexistente → responder con un **404 personalizado**.
 - Error en el servidor → responder con un **500 básico**.
10. Modularizar el código separando:
 - server.js (arranque del servidor).
 - routes.js (lógica de rutas).

4. Resultado esperado

Al finalizar el laboratorio, los alumnos dispondrán de un **servidor Node.js funcional** que:

- Atiende rutas básicas (/ , /vehiculos, /reserva).
- Gestiona formularios por POST.
- Almacena reservas en memoria.
- Sirve recursos estáticos.
- Gestiona errores simples.

Este laboratorio marca la transición entre la parte de **frontend puro (HTML, CSS, JS cliente)** y la parte de **backend** que se consolidará en el próximo laboratorio con **Express.js**.